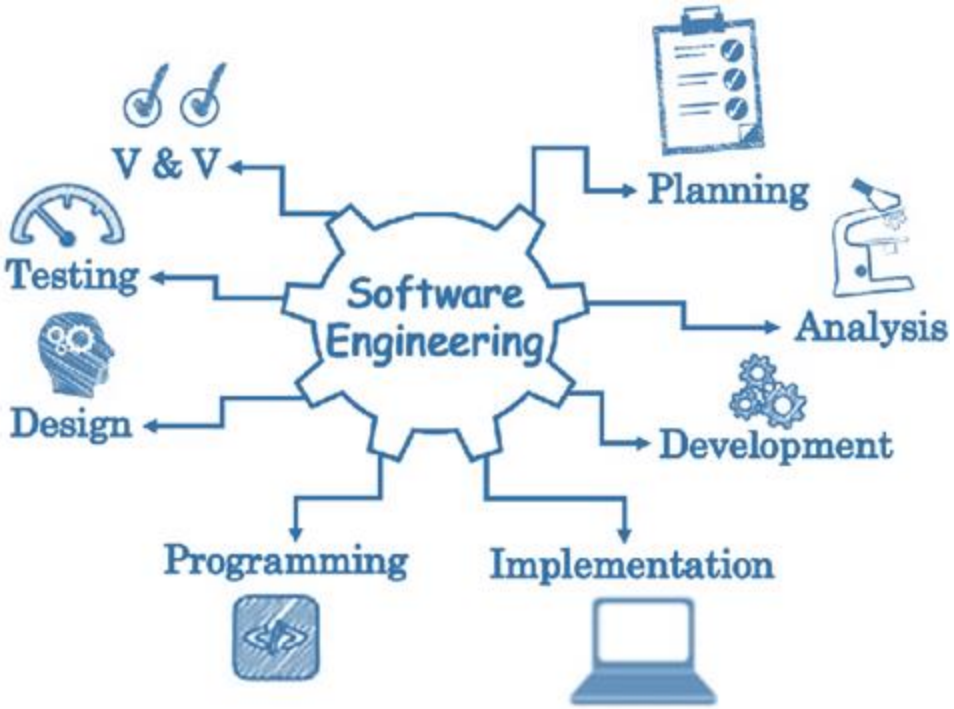
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Software Process Models and Project

2.1 Agile Development Principles

- Manifesto for Agile Software Development
- Benefits of Agile Methodologies

2.2 Scrum and Kanban Methodologies

- Scrum Roles, Artifacts, and Ceremonies
- Kanban Boards and Workflows

2.3 DevOps Culture and Principles

- Continuous Integration/Continuous Deployment (CI/CD)
- Collaboration between Development and Operations

2.4 Project Organization Concepts

- Project Organizations and Roles
- Task Assignment and Work Products
- Scheduling and Time Management

2.5 Project Communication

- Planned vs. Unplanned Communication
- Communication Mechanisms and Tools
- Meetings and Reviews

2.6 Introduction to DevOps Tools

- Overview of Jenkins, Docker, and Kubernetes

Agile Development

- To address the inefficiency of existing software development methods
- The 4 Agile Manifesto Values
- 12 Agile Principles

Find more at:
https://www.agilealliance.org/agile101/

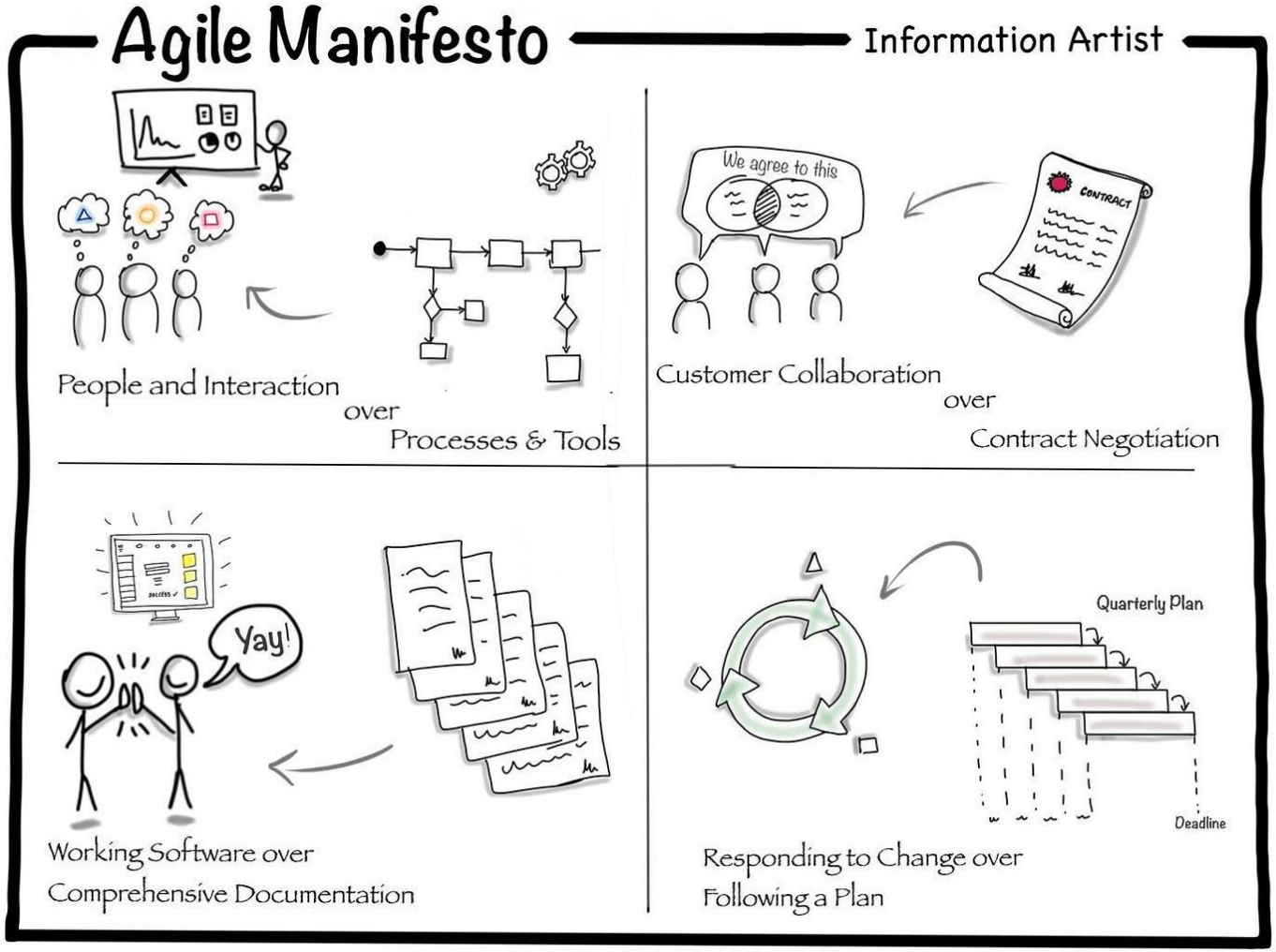
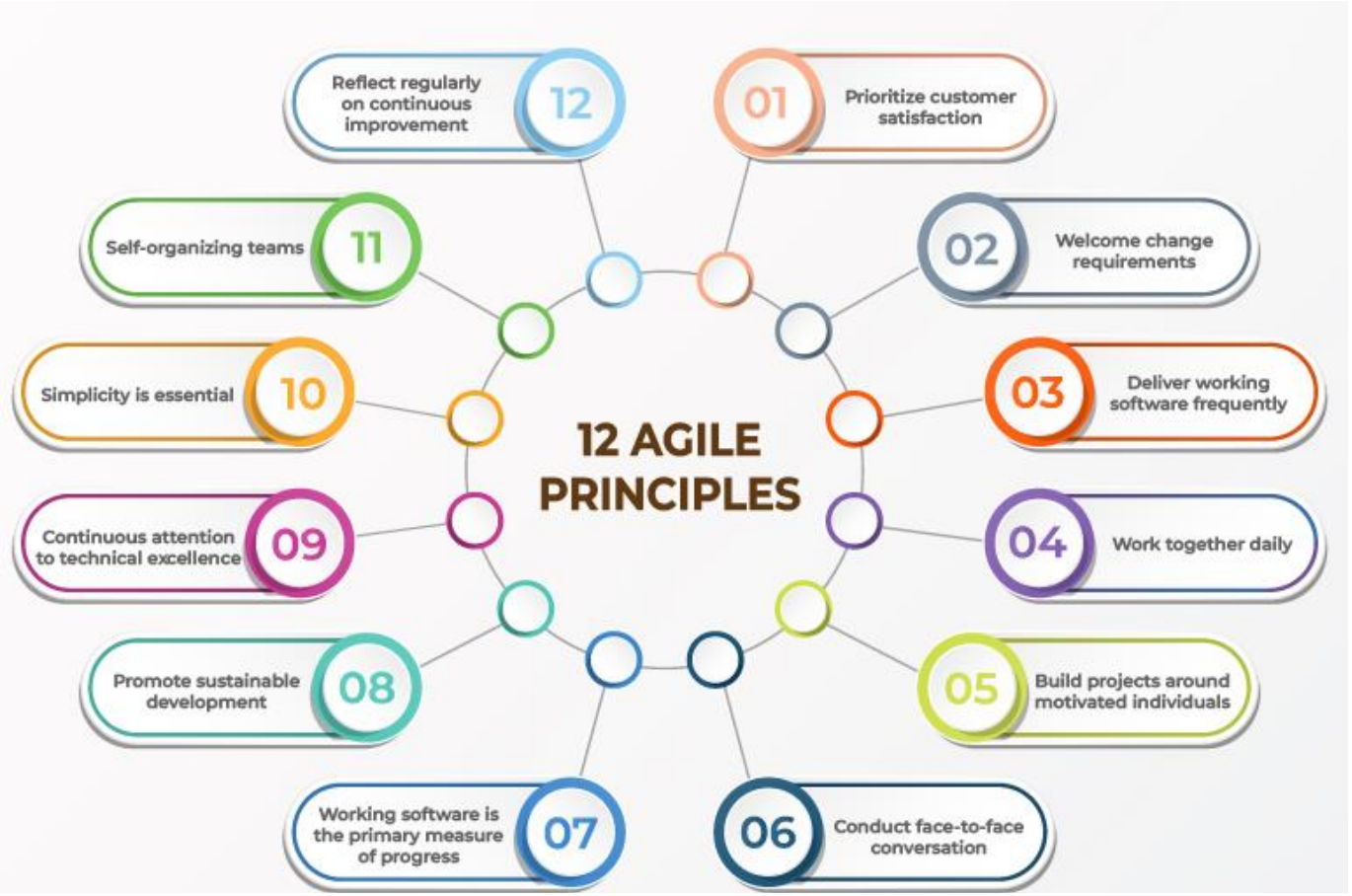


Figure: 4 Agile Values

Agile Development Principle

- The summary of 12 agile principle is :
  - *Customer satisfaction is achieved by providing **value** through software that is delivered to the customer as rapidly as possible.*
  - *To achieve this, agile developers recognize that requirements will change.*
  - *They deliver software increments frequently and work together with all stakeholders so that feedback on their deliveries is rapid and meaningful.*



Agile Development

Discussion: “Can you think of a situation in which one or more of the four “values” could get a software team into trouble?”

Situation 1: A team focuses so much on communication and teamwork that they neglect essential processes and tools. (Value 1)
Situation 2: The team prioritizes delivering software quickly but neglects creating sufficient documentation. (Value 2)
Situation 3: The team focuses heavily on satisfying customer requests through collaboration but fails to establish clear agreements or scope boundaries. (Value 3)
Situation 4: The team constantly changes priorities in response to feedback, abandoning the original plan entirely. (Value 4)

Various Agile Development Methods

- When choosing the right Agile methodology for your project, it's essential to consider the project's complexity, team size, and goals.
- **Scrum** is best suited for complex projects with large teams.
- **Kanban** is suitable for projects that require continuous delivery.
- **XP** is ideal for projects that prioritize quality,
- **Lean Software Development** is best suited for projects that require lean principles and waste elimination.



Scrum Methodology

- Conceived by Jeff Sutherland and his development team in the early 1990s.
- Further development on the Scrum methods was performed by Schwaber and Beedle.
- Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery.
- Within each framework activity, work tasks take place in a relatively short time-boxed4 period called a sprint.

•**Key Roles**:

- Product Owner (defines priorities and manages the backlog)
- Scrum Master (facilitates the process)
- Development Team (delivers increments of work).

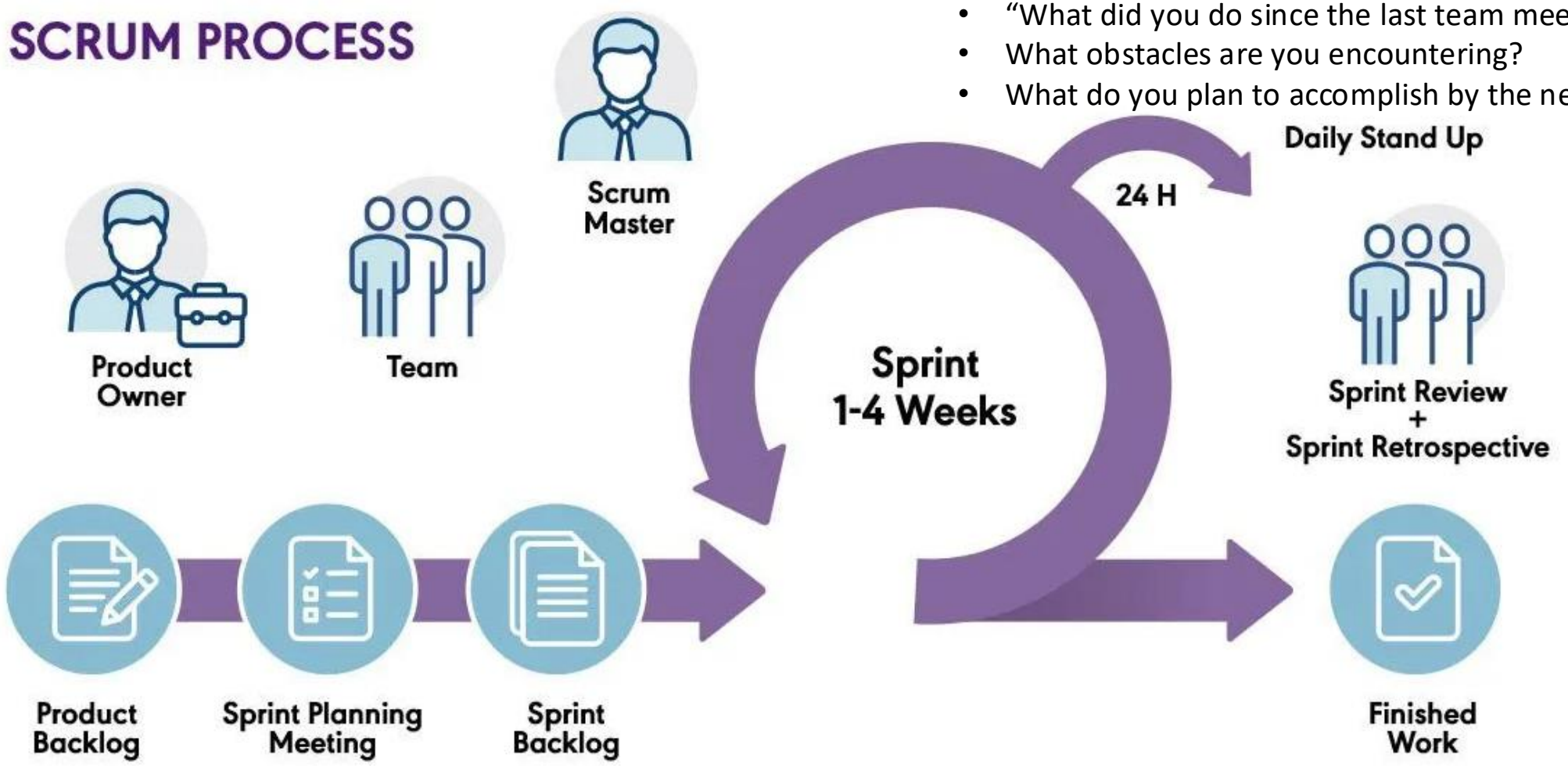
•**Artifacts**: Product Backlog, Sprint Backlog, Code Increment.

•**Ceremonies**: Sprint Planning, Daily Standup, Sprint Review, Sprint Retrospective.

Scrum Methodology

Three Questions that are asked in daily stand up meeting.

- "What did you do since the last team meeting?
- What obstacles are you encountering?
- What do you plan to accomplish by the next team meeting?"



Scrum Methodology

- **Product Backlog**
- A product backlog is a prioritized list of work for the development team that is derived from the ***product roadmap*** and its ***requirements***.
- The most important items are shown at the top of the product backlog so the team knows what to deliver first.

ToDo List

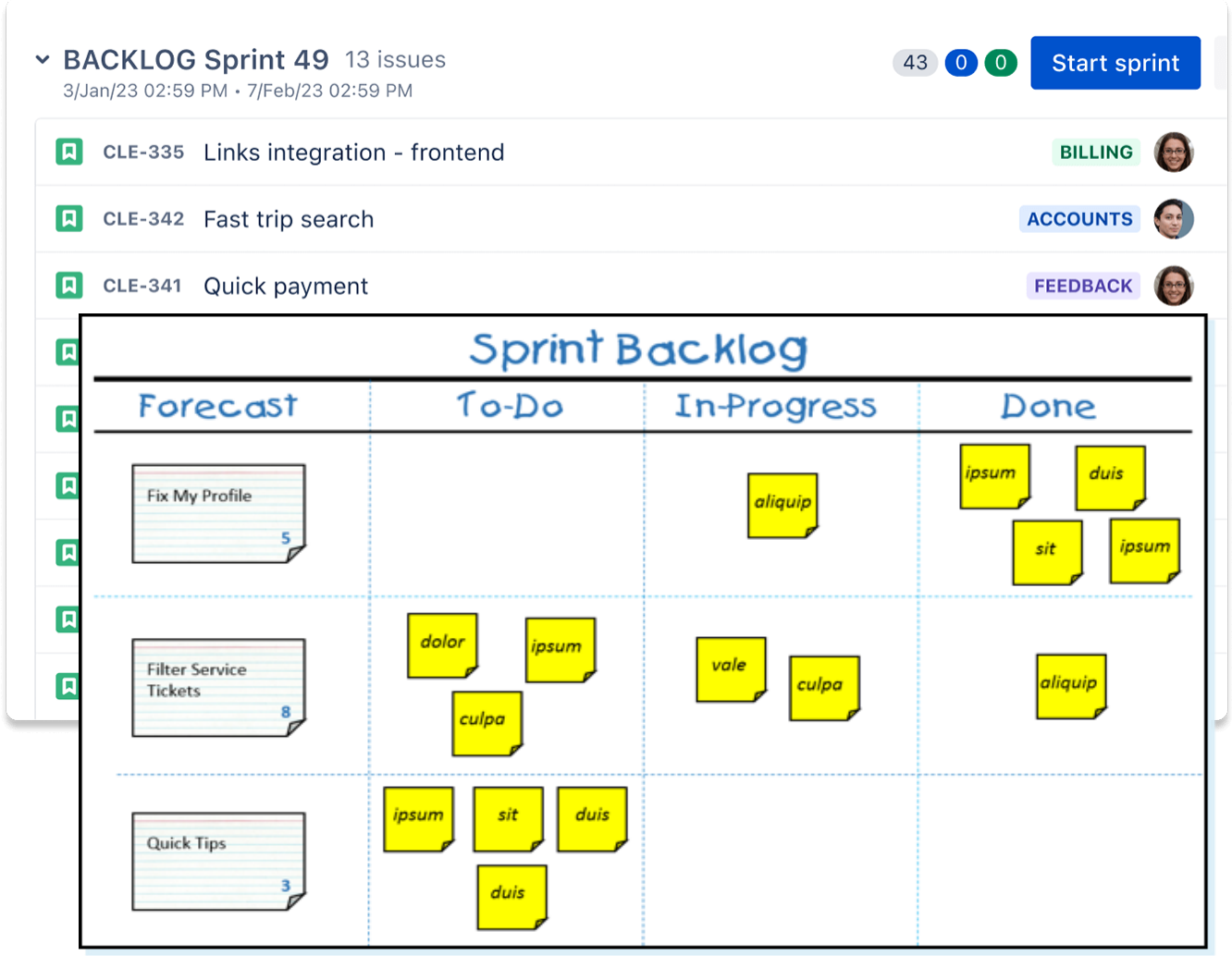
ID	Story	Estimation	Priority
7	As an unauthorized User I want to create a new account	3	1
1	As an unauthorized User I want to login	1	2
10	As an authorized User I want to logout	1	3
9	Create script to purge database	1	4
2	As an authorized User I want to see the list of items so that I can select one	2	5
4	As an authorized User I want to add a new item so that it appears in the list	5	6
3	As an authorized User I want to delete the selected item	2	7
5	As an authorized User I want to edit the selected item	5	8
6	As an authorized User I want to set a reminder for a selected item so that I am reminded when item is due	8	9
8	As an administrator I want to see the list of accounts on login	2	10
Total		**30**	

MAIN TYPES OF PRODUCT BACKLOG ITEMS

	Definition	Example
User stories	Short descriptions of desired functionality from the end-user perspective	*As an online shopper, I want to store my payment information so that I don't have to enter it repeatedly.*
Epics	Large chunks of work that can be broken down into user stories or tasks	*Account creation*
Tasks	Specific actions to be completed as part of user stories or epics	*Add social media links to the footer.*
Bug/defects and bug fixes	Issues in a product that need to be fixed	*Correct broken image links on product pages.*
Technical debt	Rework after quick, easy solutions are chosen instead of more efficient, but time-consuming, ones	*Refactor legacy authentication code.*
Enhancements and improvements	Work on existing features to make them more efficient or user-friendly	*Upgrade notification system.*
Knowledge acquisition	Additional research, skills acquisition, requirements clarification, etc.	*Learn about any legal or compliance requirements.*

Scrum Methodology

- **Sprint Backlog**
  - The Sprint Backlog is the subset of Product Backlog, composed of the ***Sprint Goal*** (why), the set of ***Product Backlog items*** selected for the Sprint (what), as well as an ***actionable plan*** for delivering the Increment (how).
  - The Sprint Backlog is a plan by and for the ***Developers***.
  - It is a highly visible, real-time picture of the work that the Developers plan to accomplish during the Sprint.



- Discussion?
  - What is the difference between Product Backlog and Sprint backlog?

Scrum Methodology

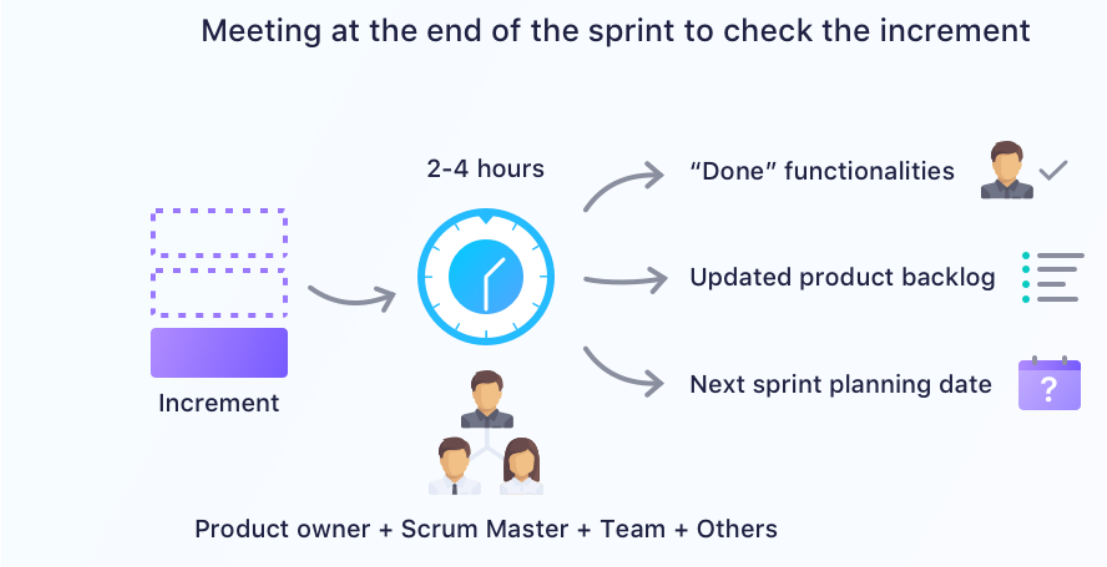
- **Ceremonies**
  - Sprint Planning Meeting
  - Daily Scrum Meeting
  - Sprint Review Meeting
  - Sprint Retrospective



- **Daily Scrum Meeting**
  - The daily Scrum meeting is a 15-minute event scheduled at the start of each workday to allow team members to synchronize their activities and make plans for the next 24 hours.
  - The Scrum master and the development team always attend the daily Scrum. Some teams allow the product owner to attend occasionally.
  - The Scrum master leads the meeting and assesses the responses from each person.
  - It is the Scrum master's task to clear obstacles presented before the next Scrum meeting if possible.
  - Also, these daily meetings lead to "knowledge socialization" [Bee99] and thereby promote a self-organizing team structure.

Scrum Methodology

- **Ceremonies**
  - Sprint Planning Meeting
  - Daily Scrum Meeting
  - Sprint Review Meeting
  - Sprint Retrospective



- **Sprint Review Meeting**
  - The sprint review occurs at the end of the sprint when the development team has judged the increment complete.
  - The Scrum master, the development team, the product owner, and selected stakeholders attend this review.
  - The primary activity is a demo of the software increment completed during the sprint.
  - The product owner may accept the increment as complete or not. If it is not accepted, the product owner and the stakeholders provide feedback to allow a new round of sprint planning to take place.

Scrum Methodology

- **Ceremonies**
  - Sprint Planning Meeting
  - Daily Scrum Meeting
  - Sprint Review Meeting
  - Sprint Retrospective



- **Sprint Retrospective**
  - The Scrum master leads the meeting and encourages the team to improve its development practices to become more effective for the next sprint.
  - The team plans ways to improve product quality by adapting its definition of “done.”
  - At the end of this meeting, the team should have a good idea about the improvements needed in the next sprint and be ready to plan the increment at the next sprint planning meeting.

Kanban Method

- The Kanban method [And16] is a lean methodology that describes methods for improving any process or workflow.
- Kanban is focused on ***change management*** and ***service delivery***.
- ***Change management*** defines the process through which a requested change is integrated into a software-based system.
- ***Service delivery*** is encouraged by focusing on understanding customer needs and expectations.
- The team members manage the work and are given the freedom to organize themselves to complete it.
- Policies evolve as needed to improve outcomes.

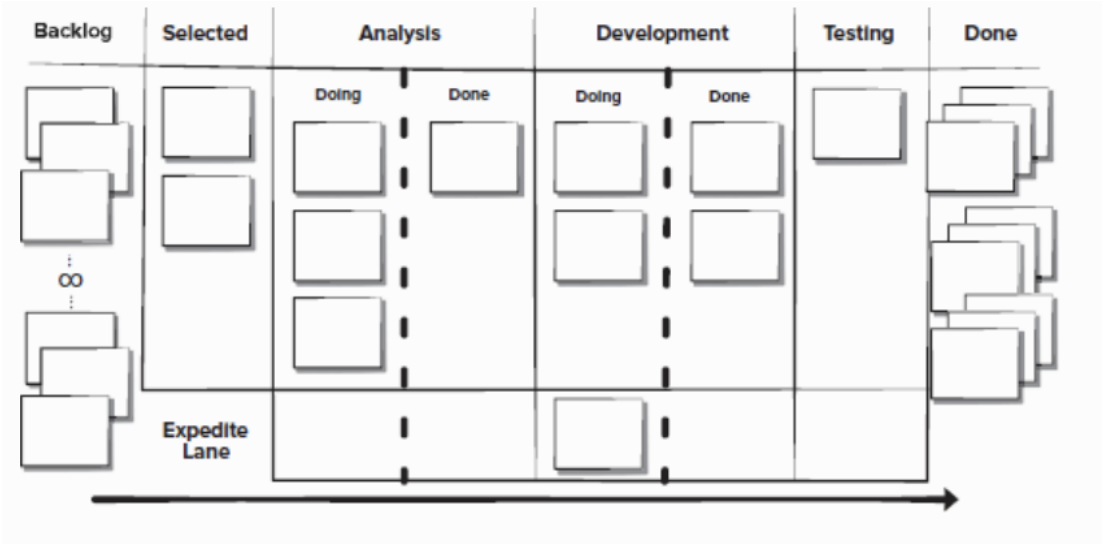


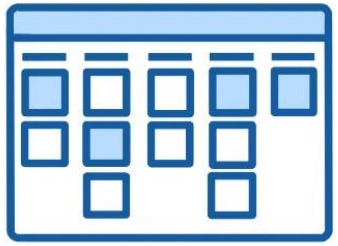
Figure: Kanban Board

Kanban Method

- **6 Core Kanban Practices**

The six foundational Kanban practices for a successful implementation include:

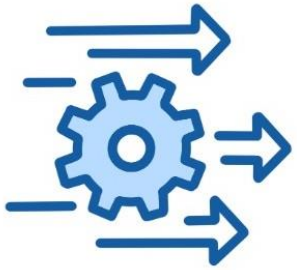
- Visualize the workflow
- Limit ‘work in progress’
- Manage flow
- Make process policies explicit
- Implement feedback loops
- Improve collaboratively



VISUALIZE



LIMIT WORK IN PROGRESS



MANAGE FLOW



MAKE POLICIES EXPLICIT



IMPLEMENT FEEDBACK LOOPS



IMPROVE COLLABORATIVELY

Kanban Method

- **Kanban Daily Stand-up**
  - The basis of the daily Kanban standup meeting is a task called “walking the board.”
  - Leadership of this meeting rotates daily.
  - The team members identify any items missing from the board that are being worked on and add them to the board.
  - The team tries to advance any items they can to “done.”
  - The goal is to advance the high business value items first.
  - The team looks at the flow and tries to identify any impediments to completion by looking at workload and risks.

- **Kanban Weekly Retrospective**
  - During the weekly retrospective meeting, process measurements are examined.
  - The team considers where process improvements may be needed and proposes changes to be implemented.
  - Kanban can easily be combined with other agile development practices to add a little more process discipline.



Discussion: In which situation the Kanban practice turn out counter productive?

No Enforced Work-In-Progress (WIP) Limits

•**Issue**: One of Kanban's core principles is limiting the amount of work in progress. If the team does not enforce WIP limits, tasks can pile up, leading to bottlenecks, multitasking, and inefficiencies.

•**Example**: A team starts multiple tasks simultaneously but struggles to finish any of them because they are spread too thin.

•**Result**: This leads to delays, burnout, and reduced quality of output.

No Workflow Defined or Improperly Defined Workflow

•**Issue**: Kanban relies on clearly defined workflows (e.g., To Do → In Progress → Done). If the workflow stages are unclear, overly complex, or not aligned with the team's processes, it can lead to confusion and inefficiency.

•**Example**: A team has ambiguous columns like "In Review," but it’s unclear who is responsible or what needs to be done.

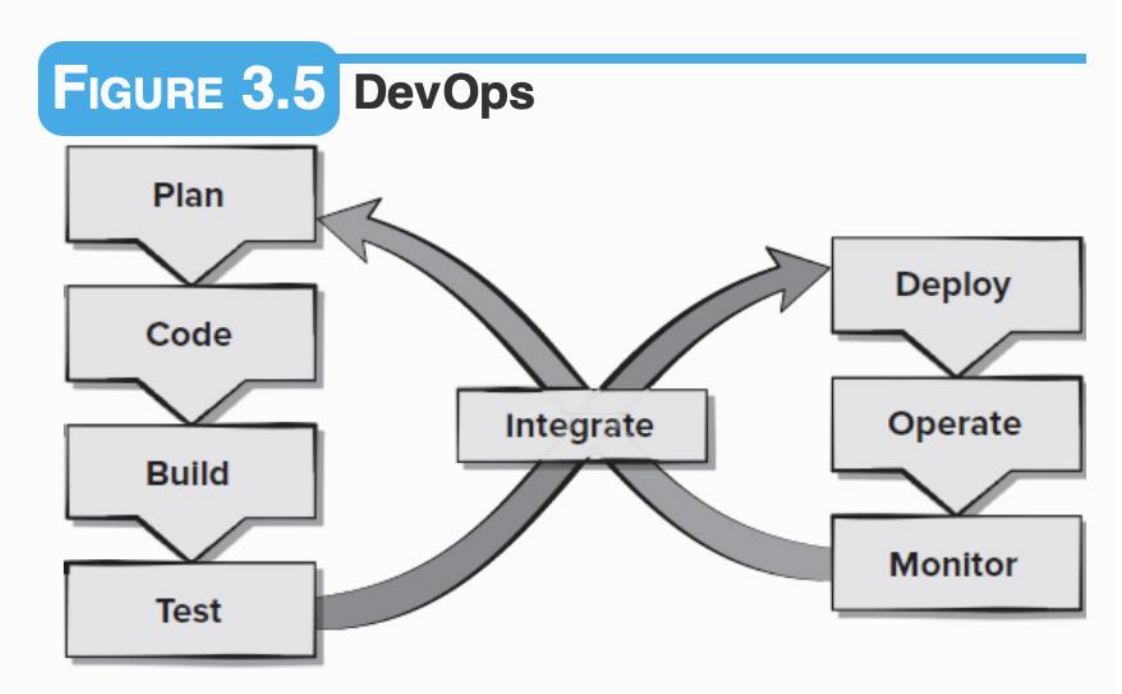
•**Result**: Tasks remain stuck in one column, leading to slow progress and frustration

Discussion

- Most agile process models recommend face-to-face communication. Yet today, members of a software team and their customers may be geographically separated from one another. Do you think this implies that geographical separation is something to avoid? Can you think of ways to overcome this problem?

DevOps

- DevOps was created by Patrick DeBois [Kim16a] to combine ***Development*** and ***Operations***.
- DevOps attempts to apply agile and lean development principles across the entire software supply chain.
- Figure 3.5 presents an overview of the DevOps workflow.
- The DevOps approach involves several stages that loop continuously until the desired product exists.
- DevOps enhances customers' experiences by reacting quickly to changes in their needs or desires.



DevOps

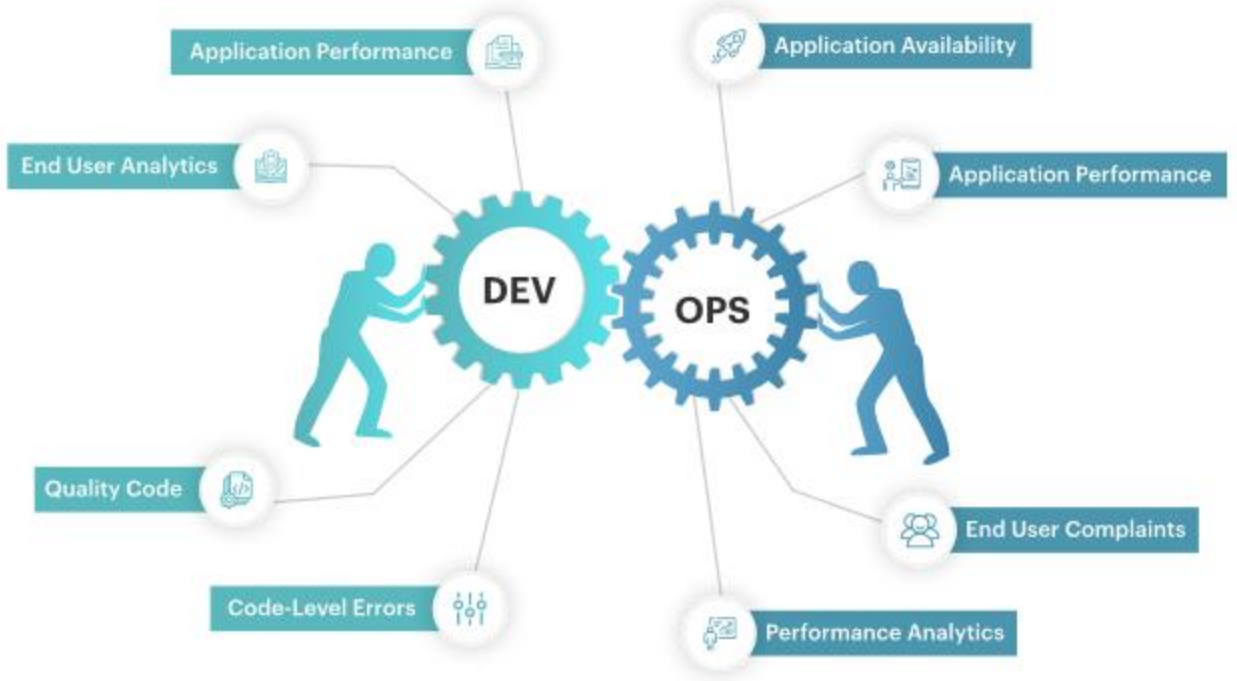
- **Continuous development.**
  - Software deliverables are broken down and developed in multiple sprints with the increments delivered to the quality assurance14 members of the development team for testing
- **Continuous testing.**
  - Automated testing tools15 are used to help team members test multiple code increments at the same time to ensure they are free of defects prior to integration.
- **Continuous integration.**
  - The code pieces with new functionality are added to the existing code and to the run-time environment and then checked to ensure there are no errors after deployment.
- **Continuous deployment.**
  - At this stage the integrated code is deployed (installed) to the production environment, which might include multiple sites globally that need to be prepared to receive the new functionality.
- **Continuous monitoring.**
  - Operations staff who are members of the development team help to improve software quality by monitoring its performance in the production environment and proactively looking for possible problems before users find them.

DevOps Culture

- At its essence, a DevOps culture involves closer collaboration and a shared responsibility between development and operations for the products they create and maintain.
- At the heart of DevOps culture is **increased transparency, communication, and** collaboration between teams that traditionally worked in siloes.
- DevOps is an organizational culture shift that emphasizes continuous learning and continuous improvement, especially through team autonomy, fast feedback, high empathy and trust, and cross-team collaboration.
- DevOps entails **shared responsibilities**. Development and operations staff should both be responsible for the success or failure of a product.
- A DevOps team culture values fast **feedback** that can help with continuous improvement of a unified development and operations team.
- **Automation** is essential to DevOps culture, since it allows great collaboration and frees up resources. Automating and integrating the processes between software development and IT teams helps them to build, test, and release software faster and more reliably.

Development and Operation Collaboration

- All the tooling and automation in the world are useless unless development and IT/Ops professionals work together.
- DevOps doesn’t solve tooling problems. It solves human problems.
- CI/CD is a cornerstone practice within the DevOps framework. It bridges the gap between development (Dev) and operations (Ops) through automation and continuous processes.
- By automating the build, test, and deployment phases, CI/CD enables rapid, reliable software releases.



Comparing Agile techniques

Scrum pros	The product owner sets priorities. The team owns decision making. Documentation is lightweight. It supports frequent updating.
Scrum cons	It is difficult to control the cost of changes. It may not be suitable for large teams. It requires expert team members.
XP pros	It emphasizes customer involvement. It establishes rational plans and schedules. There is high developer commitment to the project. There is reduced likelihood of product rejection.
XP cons	There is temptation to "ship" a prototype. It requires frequent meetings about increasing costs. It may allow for excessive changes. There is a dependence on highly skilled team members.
Kanban pros	It has lower budget and time requirements. It allows for early product delivery. Process policies are written down. There is continuous process improvement.
Kanban cons	Team collaboration skills determine success. Poor business analysis can doom the project. Flexibility can cause developers to lose focus. Developer reluctance to use measurement.
DevOps pros	There is reduced time to code deployment. The team has developers and operations staff. The team has end-to-end project ownership. There is proactive monitoring of deployed product.
DevOps cons	There is pressure to work on both old and new code. There is heavy reliance on automated tools to be effective. Deployment may affect the production environment. It requires an expert development team.

Project Organization Concepts

- Project Organizations and Roles
- Task Assignment and Work Products
- Scheduling and Time Management

Developer's Responsibilities

- Who is responsible for which part of the system?
- Which part of the system is due by when?
- Who should be contacted when a problem with a specific version of a component is discovered?
- How should a problem be documented?
- What are the quality criteria for evaluating the system?
- In which form should new requirements be communicated to developers?
- Who should be informed of new requirements?
- Who is responsible for talking to the client?

An overview of Project

- From a developer's perspective, a project consists of four components (Figure 3-1):
  - **Work product**. This is any item produced by the project, such as a piece of code, a model, or a document. Work products produced for the client are called deliverables.
  - **Schedule**. This specifies when work on the project should be accomplished.
  - **Participant**. This is any person participating in a project. Sometimes we also call the participant project member.
  - **Task.** This is the work to be performed by a project participant to create a work product.

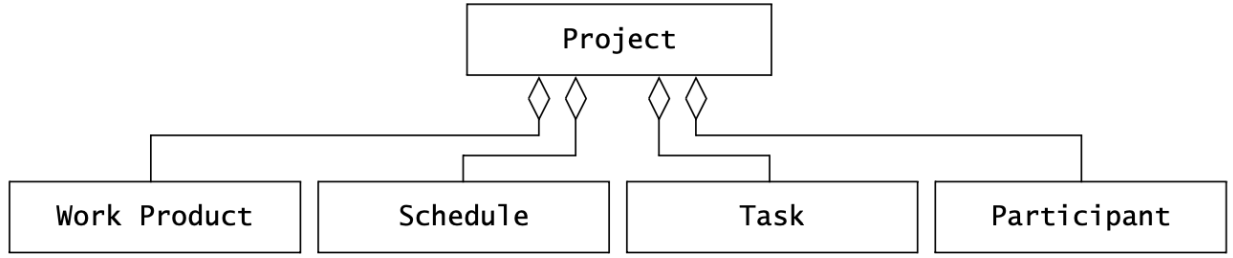


Figure 3-1 Model of a project (UML class diagram).

Source: Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java™

An overview of Project

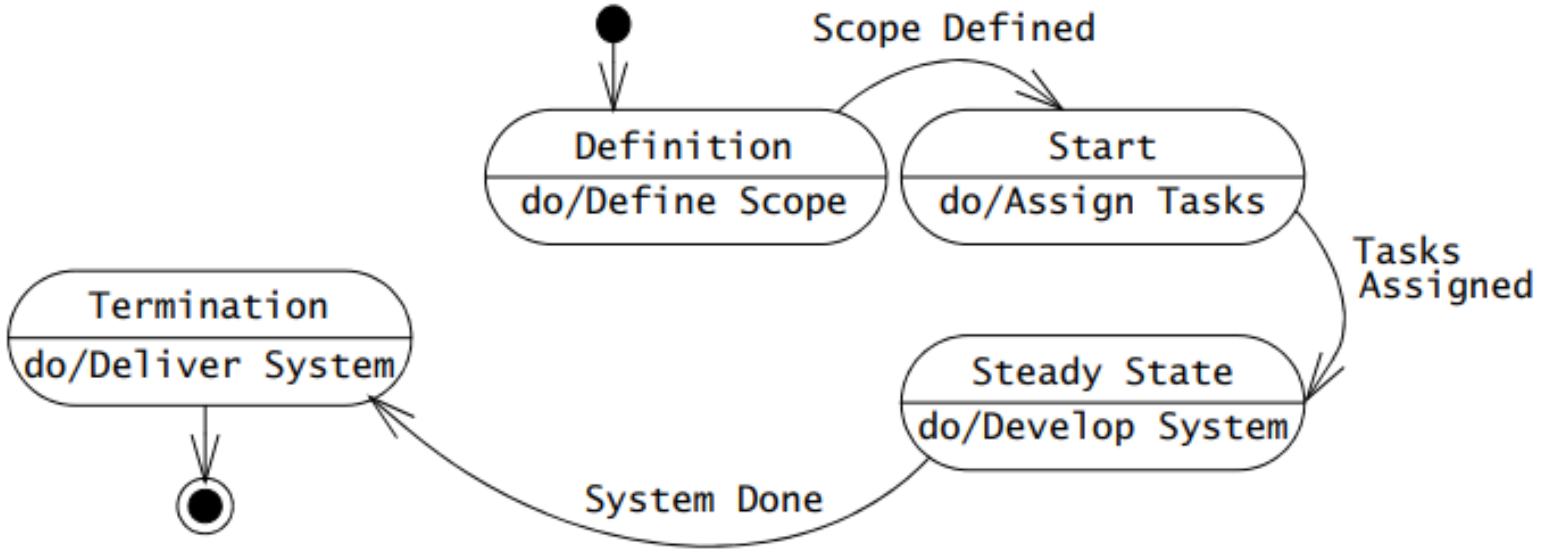


Figure 3-2 States in a software project (UML state machine diagram).

Project Organization

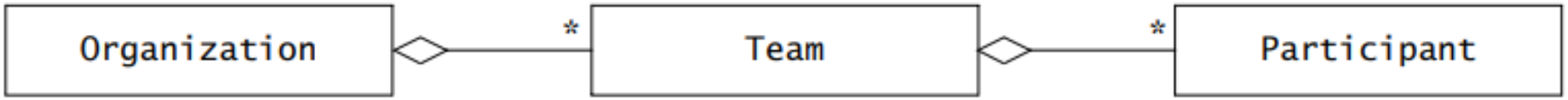


Figure 3-3 A team-based organization consists of organizational units called teams, which consist of participants or other teams (UML class diagram).

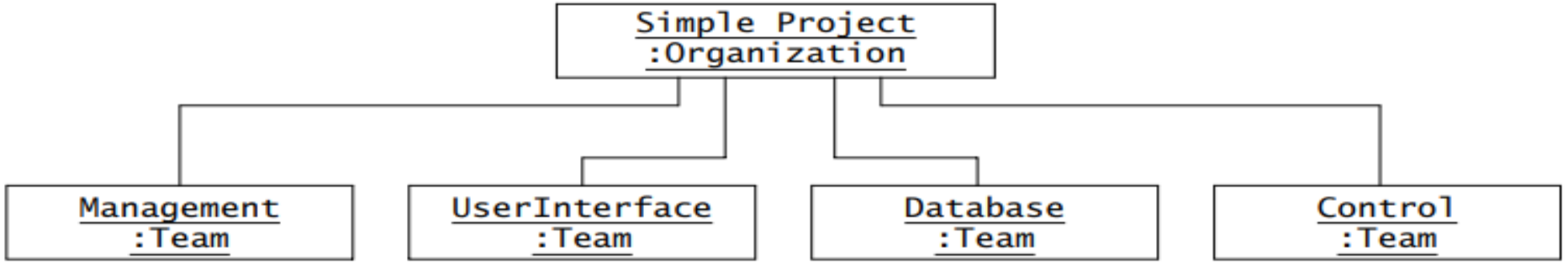


Figure 3-4 Example of a simple project organization (UML instance diagram). Reporting, deciding, and communicating are all made via the aggregation association of the organization.

Project : Types of Interaction

- **Reporting**. This type of interaction is used for reporting status information.
  - For example, a developer reports to another developer that an API (Application Programmer Interface) is ready, or a team leader reports to a project manager that an assigned task has not yet been completed.
- **Decision**. This type of interaction is used for propagating decisions.
  - For example, a team leader decides that a developer has to publish an API, a project manager decides that a planned delivery must be moved up in time. Another type of decision is the resolution of an issue.
- **Communication**. This type of interaction is used for exchanging all the other types of information needed for decision or status. Communication can take many flavors.
  - Examples are the exchange of requirements or design models or the creation of an argument to support a proposal. An invitation to eat lunch is also a communication.

Project : Hierarchical Reporting structure

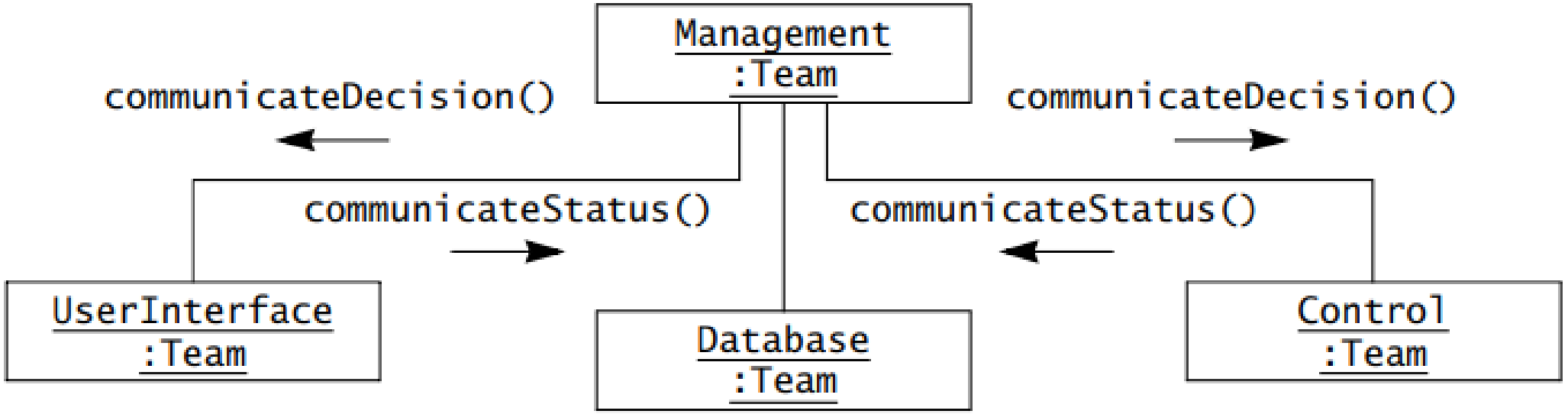


Figure 3-5 Example of reporting structure in a hierarchical organization (UML communication diagram). Status information is reported to the project manager, and corrective decisions are communicated back to the teams by the team leaders. The team leaders and the project manager are called the management team.

Project : Liaison-based Communication structure

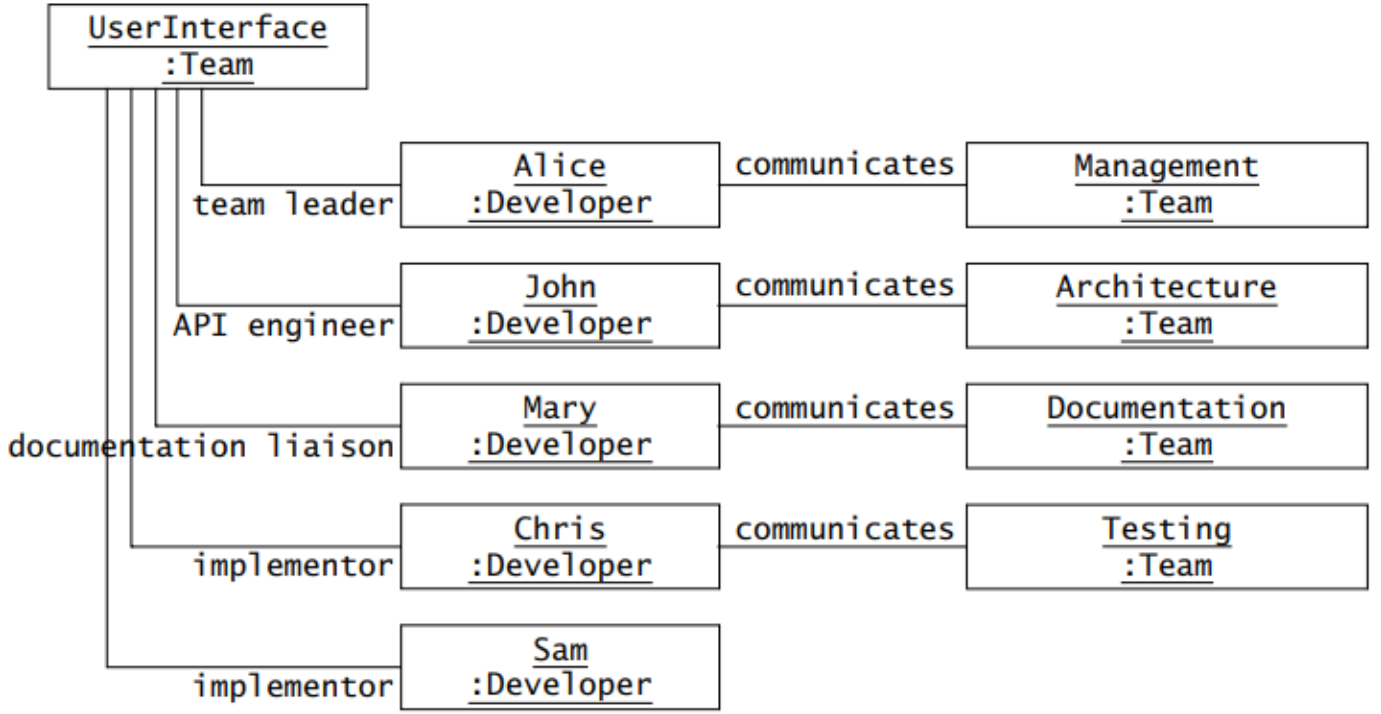
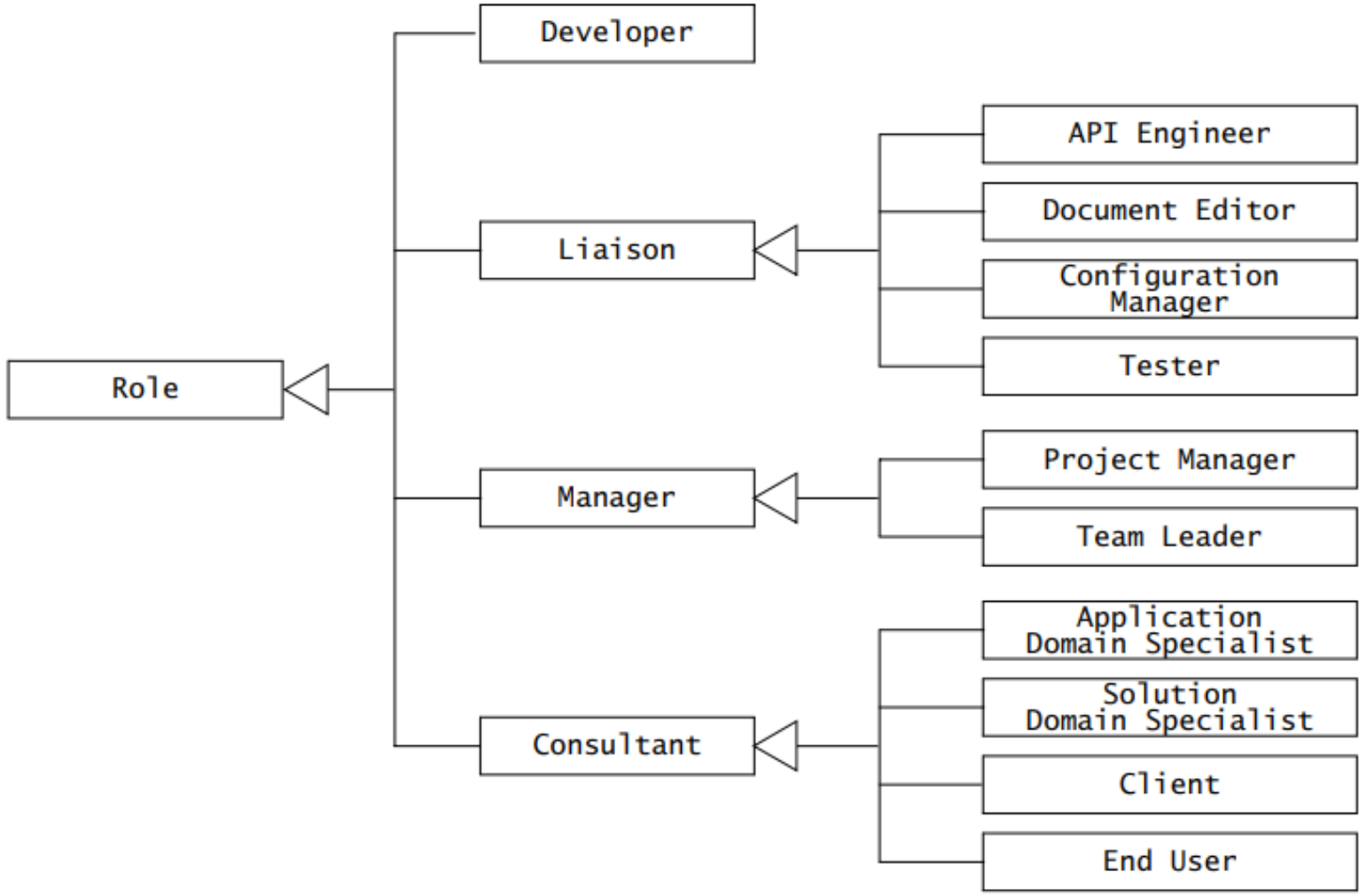


Figure 3-6 Examples of a liaison-based communication structure (UML object diagram). The team is composed of five developers. Alice is the team leader, also called the liaison to the management team. John is the API engineer, also called the liaison to the architecture team. Mary is the liaison to the documentation team. Chris and Sam are implementors and interact with other teams only informally.

Role


Role
Developer
Liaison
API Engineer
Document Editor
Configuration Manager
Tester
Manager
Project Manager
Team Leader
Consultant
Application Domain Specialist
Solution Domain Specialist
Client
End User

Role: Some Examples

Role	Responsibilities
System architect	The system architect ensures consistency in design decisions and interface styles. The system architect ensures the consistency of the design in the configuration management and testing teams, in particular in the formulation of the configuration management policy as well as the system integration strategy. This is mainly an integration role consuming information from each subsystem team.
Object designer	The object designer is responsible for the interface definition of the assigned subsystem. The interface has to reflect the functionality already assigned to the subsystem and to accommodate the needs of the dependent subsystems. When functionality is traded off with other subsystems, resulting in subsystem changes, the object designer is responsible for propagating changes back to the subsystem team.
Implementor	The implementor is responsible for the coding of a class or a number of classes associated with the subsystem.
Tester	A tester is responsible for evaluating that each subsystem works as specified by the object designer. Often, development projects have a separate team responsible only for testing. Separating the roles of implementor and tester leads to more effective testing.

Tasks and Work Product

- **Task**: A well-defined work assignment for a role.
- **Activity**: A group of related tasks.
- **Examples**
  - Role of a Project Manager/Team Leader:
    - Assign tasks to roles.
    - Monitor task progress and completion.
  - Participant's Role:
    - Carry out the assigned task.
- Work Product:
  - A tangible item resulting from a task.
  - Examples:
    - Object model
    - Class diagram
    - Source code
    - Documents or parts of documents

Work Products, Deliverables, and Work Packages

- Deliverable:
  - A work product delivered to the client.
  - Includes:
    - Software system
    - Documentation
- Internal Work Products:
- Work products not visible to the client.
- Work Package:
  - Describes work needed to complete a task or activity.
  - Components of a Work Package:
    1. Task name and description
    2. Resources needed
    3. Dependencies:
       - Inputs: Work products from other tasks
       - Outputs: Work products from the current task
    4. Links to other tasks

SCHEDULING

- Split project into tasks (= create a Work Break Down Structure (WBS))
- Estimate time and resources required to complete each task.
- Organize tasks concurrently to make optimal

  use of workforce.
- Minimize task dependencies to avoid delays

  caused by one task waiting for another to complete.
- Dependent on project managers intuition and experience.

SCHEDULING TECHNIQUES

- Mathematical Analysis
  - Network Diagrams
    - ➢ critical path method (CPM)
    - ➢ Program evaluation and review technique (PERT)

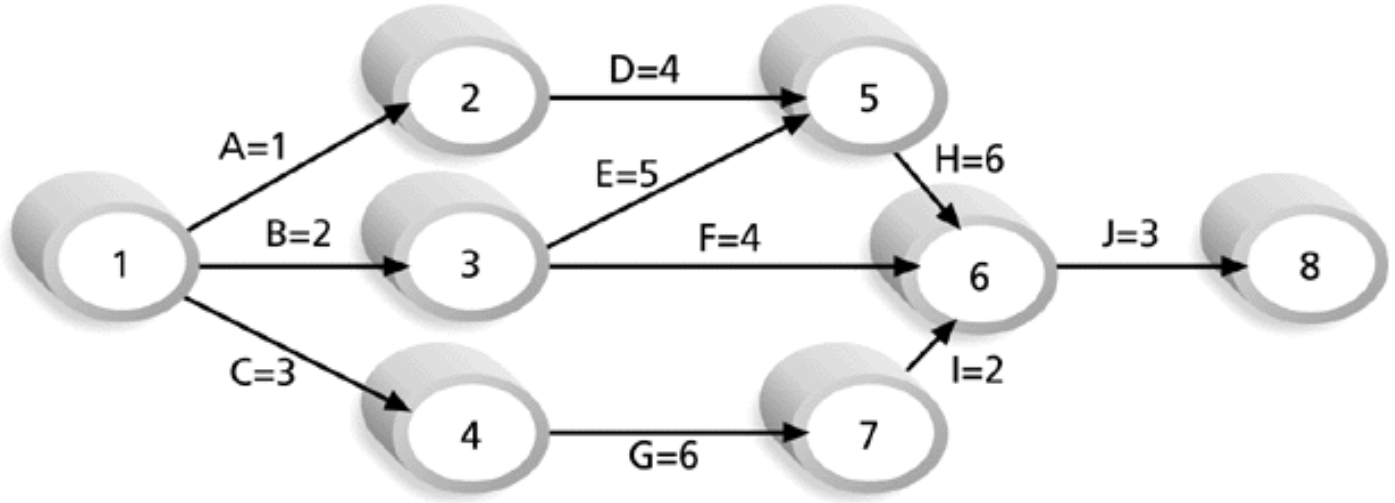
- Bar Charts
  - Milestone Chart
  - Gantt Chart/Time-line chart

Critical Path Method

- CPM is a project network analysis technique used to predict total project duration
- A critical path for a project is the series of activities that determines the *earliest time* by which the project can be completed
- The critical path is the *longest path* through the network diagram and has the least amount of slack or float

Critical Path Method

- First develop a good project *network diagram*
- Add the *durations* for all activities on each path through the project network diagram
- The *longest path* is the critical path



Note: Assume all durations are in days.

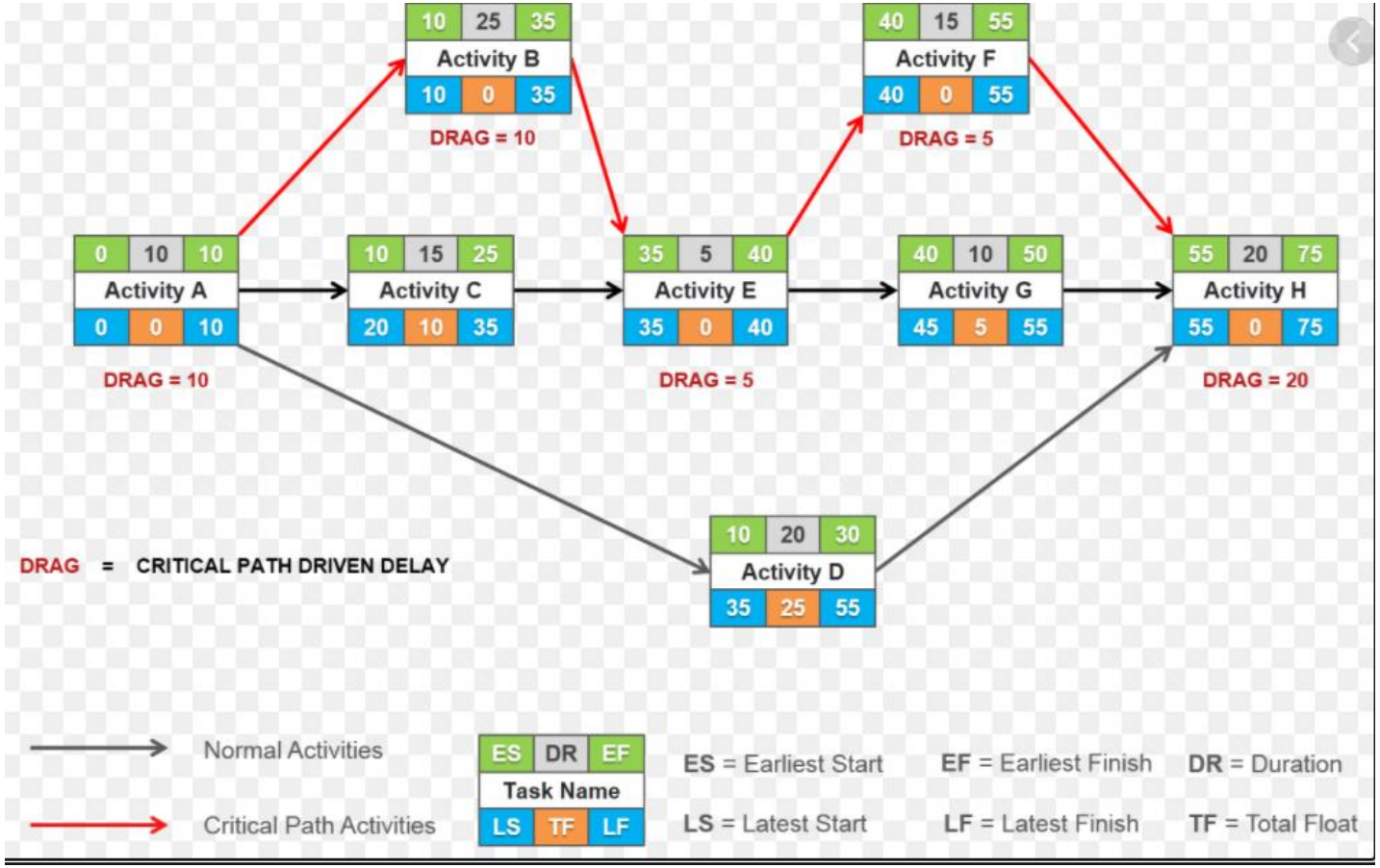
Path 1:	A-D-H-J	Length = 1+4+6+3 = 14 days
Path 2:	**B-E-H-J**	**Length = 2+5+6+3 = 16 days**
Path 3:	B-F-J	Length = 2+4+3 = 9 days
Path 4:	C-G-I-J	Length = 3+6+2+3 = 14 days

Since the critical path is the longest path through the network diagram, Path 2, B-E-H-J, is the critical path for Project X.

Figure: Determining the Critical Path for Project X

PERT

- The **Program Evaluation and Review Technique (PERT)** is a project management tool used to plan, schedule, and control complex projects.
- It provides a systematic approach for analyzing and representing the tasks required to complete a project, focusing on time estimates and interdependencies.



Key Features of PERT

- **Project Breakdown:**
  - Divides a project into smaller, manageable tasks or activities.
- **Dependency Mapping:**
  - Defines the sequence and relationships (dependencies) between tasks.
- **Time Estimation:**
  - Uses three time estimates for each task to account for uncertainty:
    - **Optimistic Time (O):** The shortest possible time to complete a task.
    - **Most Likely Time (M):** The time to complete a task under normal conditions.
    - **Pessimistic Time (P):** The longest possible time if delays occur.
- **Expected Time (TE):**
  - Calculated using the formula: $TE = (O+4M+P)/6$

Network Diagram:

A graphical representation of tasks and their dependencies, often in the form of a **PERT Chart**.

- **Critical Path Identification:**
  - Identifies the longest path through the network (critical path), determining the shortest time to complete the project.

Steps in PERT Analysis

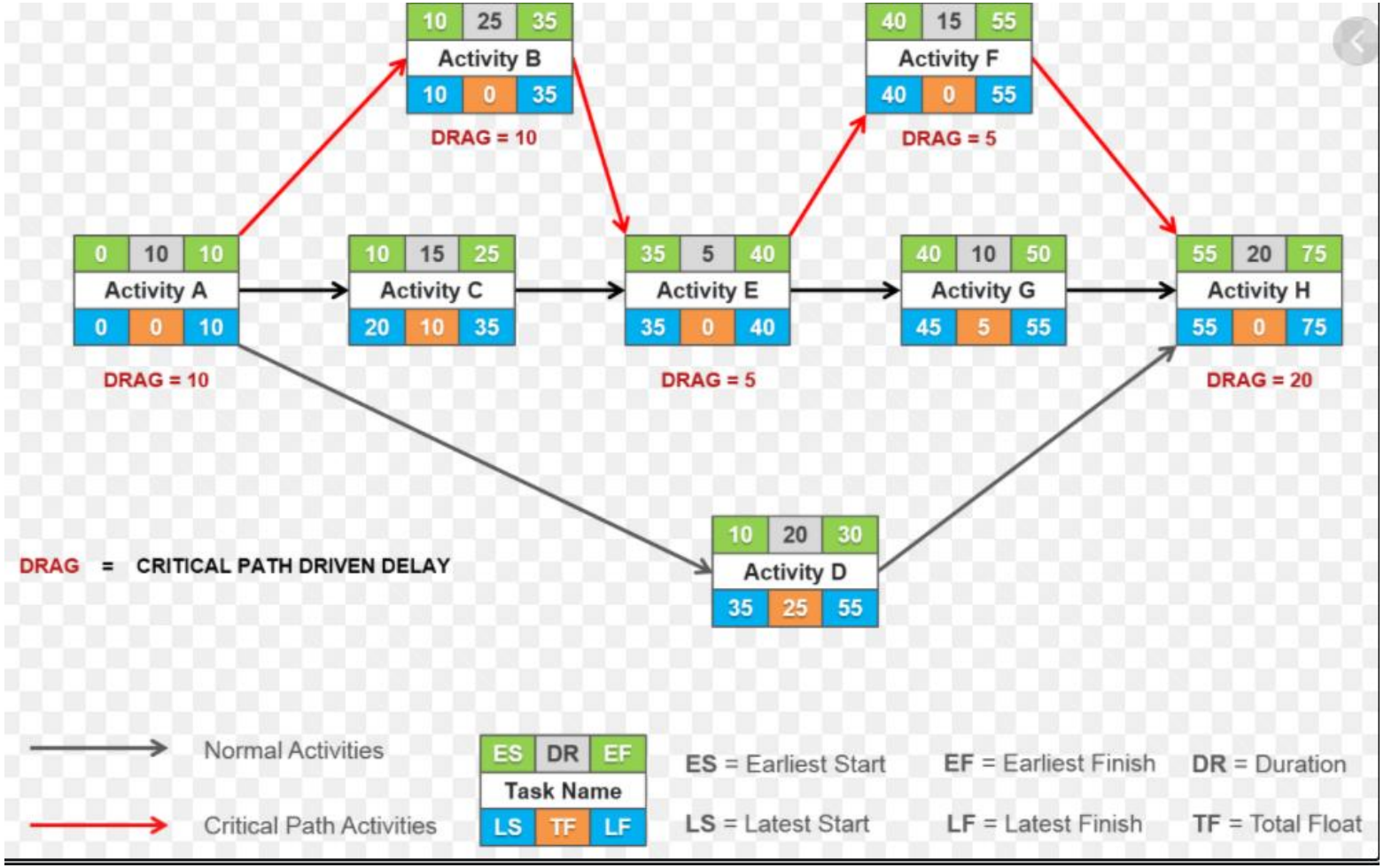
1. **Define Tasks and Milestones:**
   List all tasks required to complete the project.
2. **Determine Task Dependencies:**
   Identify the order in which tasks need to be performed.
3. **Estimate Task Durations:**
   For each task, provide the optimistic, most likely, and pessimistic time estimates.
4. **Draw the PERT Chart:**
   Use nodes to represent tasks and arrows to indicate dependencies.
5. **Calculate Expected Time for Each Task:**
   Apply the TE formula to estimate task durations.
6. **Identify the Critical Path:**
   Determine the sequence of tasks that cannot be delayed without impacting the overall project timeline.
7. **Update the PERT Chart:**
   Monitor progress and revise estimates as necessary during project execution.

Steps in PERT Analysis

Activity	Duration (in weeks)	Precedents
A	6	–
B	4	–
C	3	A
D	4	B
E	3	B
F	10	–
G	3	E, F
H	2	C, D

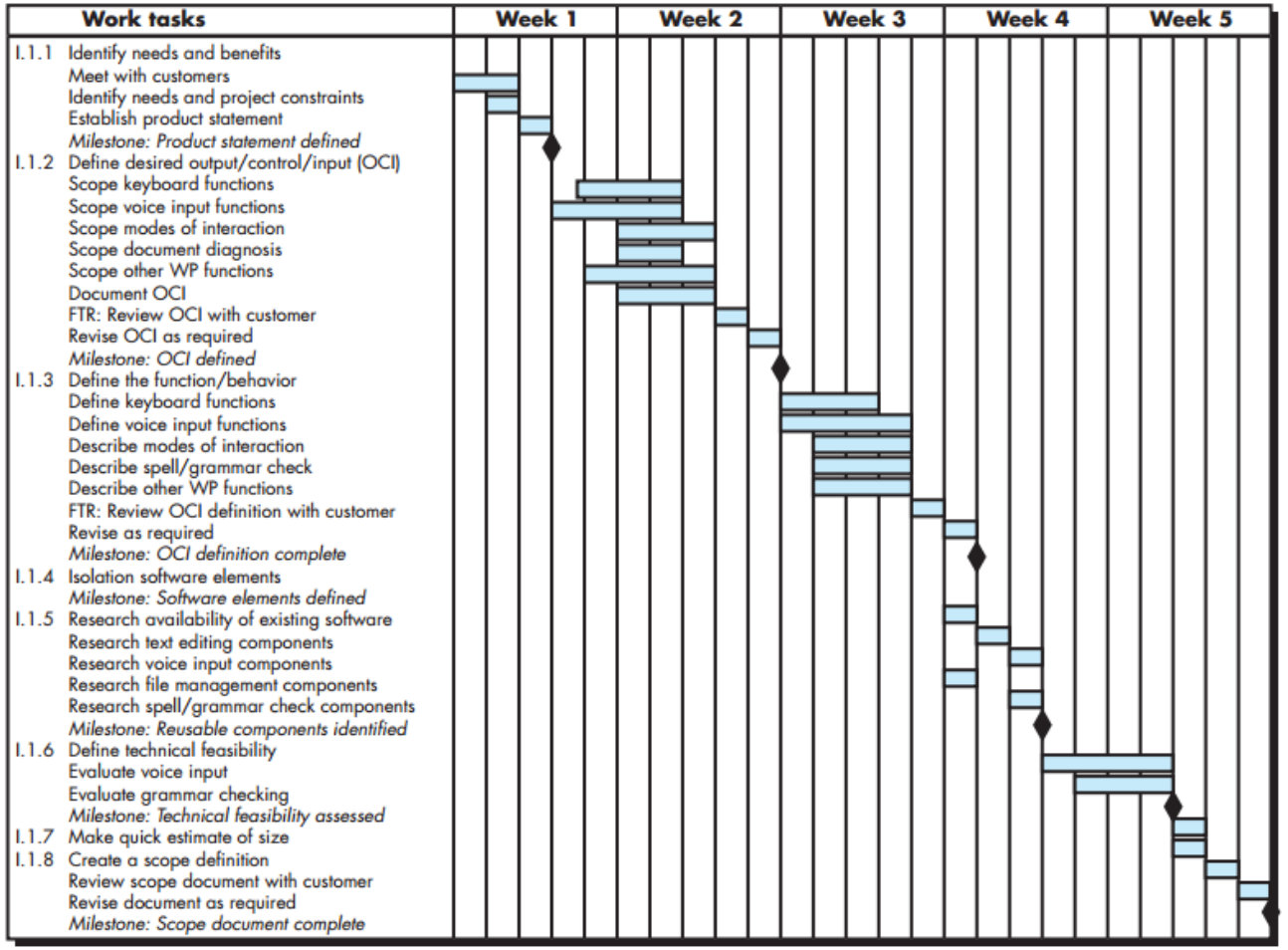
Earliest Start	Duration	Earliest Finish
Activity Label		
Latest Start	Float	Latest Finish

Image- https://www.project-risk-manager.com/blog/pert-vs-cpm/



Timeline chart/Gantt Chart

FIGURE 34.3 An example time-line chart



Project Communication

Planned Vs Unplanned Communications?

Planned vs Unplanned Communications

- Planned Communications Examples
  - problem inspection, during which developers gather information from the problem statement, the client, and the user about their needs and the application domain
  - status meetings, during which teams review their progress
  - peer reviews, during which team members identify defects and find solutions in preliminary work products
  - client and project reviews, during which the client or project members review the quality of a work product, in particular deliverables
  - releases, during which project participants make available to the client and end users versions of the system and its documentation.

Planned vs Unplanned Communications

- Planned Communications Events
  - Problem presentation
  - Client reviews
  - Project reviews
  - Peer reviews
  - Status reviews
  - Brainstorming
  - Releases
  - Postmortem reviews.

Planned vs Unplanned Communications

- Unplanned Communications Examples
  - Requests for clarification, during which participants request specific information from others about the system, the application domain, or the project
  - Requests for change, during which participants describe problems encountered in the system or new features that the system should support
  - Issue resolution, during which a conflict between different stakeholders is identified, solutions explored and negotiated, and a resolution agreed upon.

Planned vs Unplanned Communications

- Unplanned Communications Events
  - Requests for clarification
  - Requests for changes
  - Issue resolution.

Communication Mechanism

- Synchronous
- Asynchronous

Synchronous Communication

Mechanism	Supported communication events
Hallway conversations	Request for clarification Request for change
Questionnaires and structured interviews	Problem definition Postmortem review
Meetings (face-to-face, telephone, video)	Problem definition Client review Project review Peer review Status review Postmortem review Brainstorming Issue resolution
Synchronous groupware	Client review Project review Peer review Brainstorming Issue resolution

Asynchronous Communication

Mechanism	Supported communication events
Electronic mail	Change request Brainstorming
Newsgroups	Change request Brainstorming
World Wide Web	Release Asynchronous peer reviews Change request Brainstorming
Lotus Notes	Release Asynchronous peer reviews Change request Brainstorming

Communication Mechanisms and Tools

1. Synchronous Communication Tools
   - Meetings: In-person, virtual (Zoom, Microsoft Teams, Google Meet)
   - Chat Applications: Slack, Microsoft Teams, Discord
   - Phone/Voice Calls: Traditional phone calls or VoIP services
   - Pair Programming Platforms: Visual Studio Live Share, CodeTogether
2. Asynchronous Communication Tools
   - Email: Outlook, Gmail
   - Discussion Forums: GitHub Discussions, Slack Threads, Microsoft Teams Channels
   - Document Sharing Platforms: Google Docs, OneDrive, SharePoint

Communication Mechanisms and Tools

3. Project Management and Collaboration Tools
    - Issue Tracking and Task Boards: Jira, Trello, Asana, ClickUp
    - Collaboration Software: Confluence, Notion, Miro, Mural
    - Version Control: GitHub, GitLab, Bitbucket

4. File and Documentation Management
    - File Sharing: Dropbox, Google Drive, OneDrive
    - Documentation Tools: Confluence, Notion, Google Docs, Markdown-based tools

Communication Mechanisms and Tools

5. Review and Feedback Tools
    - Code Review Platforms: GitHub Pull Requests, GitLab Merge Requests
    - Testing and Bug Tracking Tools: Bugzilla, TestRail, Zephyr
    - Feedback Forms: Google Forms, Typeform
6. Reporting and Visualization Tools
    - Dashboards and Reporting: Tableau, Power BI, Jira Dashboards
    - Time Tracking Tools: Toggl, Harvest, Clockify
7. Hybrid Communication Tools
    - All-in-One Tools: Microsoft Teams, Slack, Zoom (support chat, calls, and file sharing)

SOFTWARE MEETINGS

Importance and Roles in Face-to-Face Meetings

Key Benefits of Meetings:

- Enable participants to share, review, and negotiate solutions
- Effective for resolving issues and building consensus

Challenges of Meetings:

- High resource cost
- Difficult to manage

Key Roles in Meetings:

- **Facilitator:**
  - Organizes and guides the meeting
  - Prepares and shares the agenda before the meeting
- **Minute Taker:**
  - Records the meeting notes and organizes them for distribution
  - Enables sharing outcomes with absent members
- **Timekeeper:**
  - Tracks time during the meeting
  - Ensures the agenda is followed, and discussions stay on track

Meeting Structure and Distributed Meetings

Meeting Agenda Structure:

1. **Header**: Meeting location, time, and participants
2. **Reports**: Items participants will report on
3. **Discussion Items**: Issues to be discussed, including allocated time per item

Meeting Minutes Structure:

- **Header**: Actual meeting details
- **Information Sharing**: Key information shared during the meeting
- **Decisions**: Decisions made or unresolved
- **Action Items**: Follow-up actions for participants

Distributed Meetings:

- Use **teleconferencing** or **video conferencing** to reduce costs
- Challenges: Lower bandwidth, reliability, and communication quality
- **Well-structured agenda** and familiarity with participant voices are crucial

Meeting Structure and Distributed Meetings

Header information identifying the meeting and audience	**When and Where** **Date**: 1/30 **Start**: 4:30 P.M. **End**: 5:30 P.M. **Room**: WH, 3420	**Role** **Primary Facilitator**: Peter **Timekeeper**: Dave **Minute Taker**: Ed
Desired outcome of the meeting	**1. Objective** Resolve any requirements issues that prevent us from starting prototyping.	
Action items to be reported on	**2. Status [Allocated Time: 15 minutes]** Dave: State of command parsing code	
Issues scheduled to be discussed (and resolved) during the meeting	**3. Discussion items [Allocated Time: 35 minutes]** 3.1 How to deal with arbitrarily formatted input data sets? 3.2 How to deal with output data? 3.3 Command parsing code (modifiability, backward compatibility)	
The wrap-up period is the same for all meetings	**4. Wrap up [Allocated Time: 5 minutes]** 4.1 Review and assign new action items 4.2 Meeting critique	

Figure 3-21 An example of a meeting agenda.

Open-ended meetings take more time than necessary.	**When and Where** **Date**: 1/30 **Start**: 4:30 P.M. **End**: open **Room**: WH 3420	**Role** **Primary Facilitator**: Peter **Timekeeper**: Dave **Minute Taker**: Ed
This objective is difficult to achieve and cannot be verified.	**1. Objective** Resolve open issues	
Lack of context: what were Dave's action items?	**2. Status [Allocated Time: 15 minutes]** Dave: Dave's action items	
Lack of content: what are the current issues in each of these activities?	**3. Discussion items [Allocated Time: 35 minutes]** 3.1 Requirements issues 3.2 Design issues 3.3 Implementation issues	
	4. Wrap up [Allocated Time: 5 minutes] 4.1 Review and assign new action items 4.2 Meeting critique	

Figure 3-22 An example of poor meeting agenda.

Meeting Structure and Distributed Meetings

Header information identifying the meeting and audience	**When and Where** **Date**: 1/30 **Start**: 4:30 P.M. **End**: 6:00 P.M. **Room**: WH 3420	**Role** **Primary Facilitator**: Peter **Timekeeper**: Dave **Minute Taker**: Ed **Attending**: Ed, Dave, Mary, Peter, Alice
Verbatim from agenda	**1. Objective** ...	
Summary of the information that was exchanged	**2. Status** ...	
Record of issue discussion and resolution	**3. Discussion** 3.1 Command parsing code is a 1200–1300 line if statement. This makes it fairly hard to add new commands or to modify existing commands without breaking backward compatibility with existing clients. Proposals: 1) Restructure the command parsing code by assigning one object per kind of command. 2) Pass all command arguments by name. The latter would make it easier to maintain backward compatibility. On the other hand, this would increase the size of the commands, thus increasing the size of the command file. Resolution: Restructure code for now. Revisit this issue if backward compatibility is really an issue (the calling code might be rewritten anyway). See AI[1]. ... *Discussion of the other issues omitted for brevity*	
Additions and modifications to the task plan	**4. Wrap up** AI[1] For: Dave. Revisit command parsing code. Emphasis on modularity. Coordinate with Bill from the database group (who might assume backward compatibility). ... *Other action items and meeting critique omitted for brevity* ...	

Figure 3-23 An example of meeting minutes.

SOFTWARE REVIEWS

SOFTWARE REVIEWS

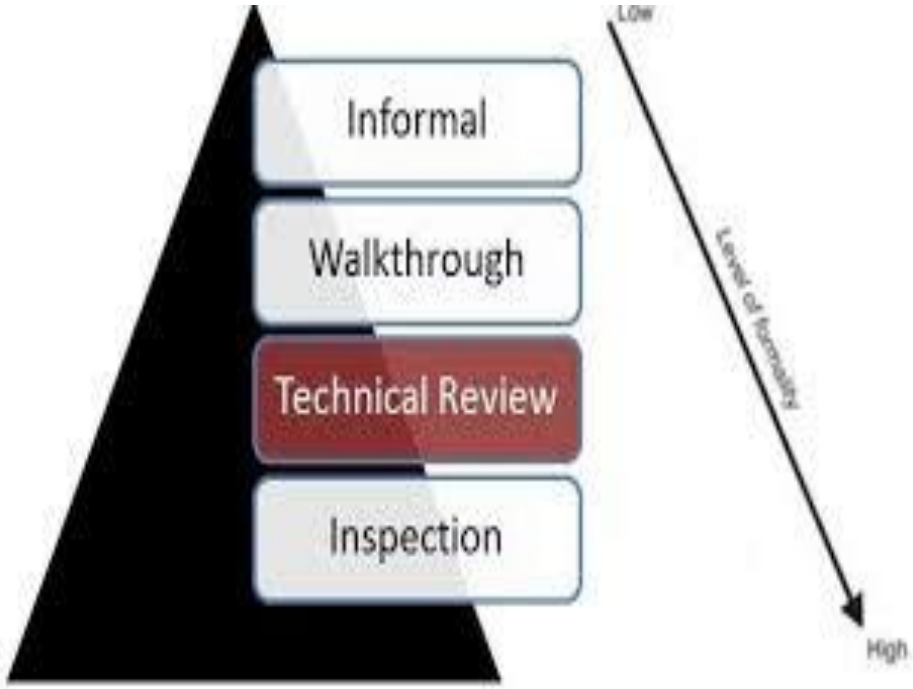
- A systematic inspection of a software by one or more individuals who work together to find and resolve errors and defects in the software during the early stages of Software Development Life Cycle (SDLC).

- Purpose
  - serves to uncover errors in analysis, design, coding, and testing.

- Why
  - To improve the productivity of the development team.
  - To make the testing process time and cost effective.
  - To make the final software with fewer defects.
  - To eliminate the inadequacies.

Types of Reviews

1. Informal reviews
   - informal meeting and informal desk checking
2. Formal reviews
   - Walkthrough, technical reviews, inspection



Formal Technical Review

Objectives of FTR:

- To uncover errors in function, logic, or implementation
- To verify the software under review meets its requirements
- To ensure that the software has been represented according to predefined standards
- To develop software in a uniform manner
- To make projects more manageable

Purposes of FTR:

- Serves as a training ground for junior engineers
- Promote backup and continuity

Formal Technical Review (FTR)

Review meeting's constraints

- 3-5 people involved in a review
- Advanced preparation (no more than 2 hours for each person)
- The duration of the review meeting should be less than 2 hours
- Focus on a specific part of a software product

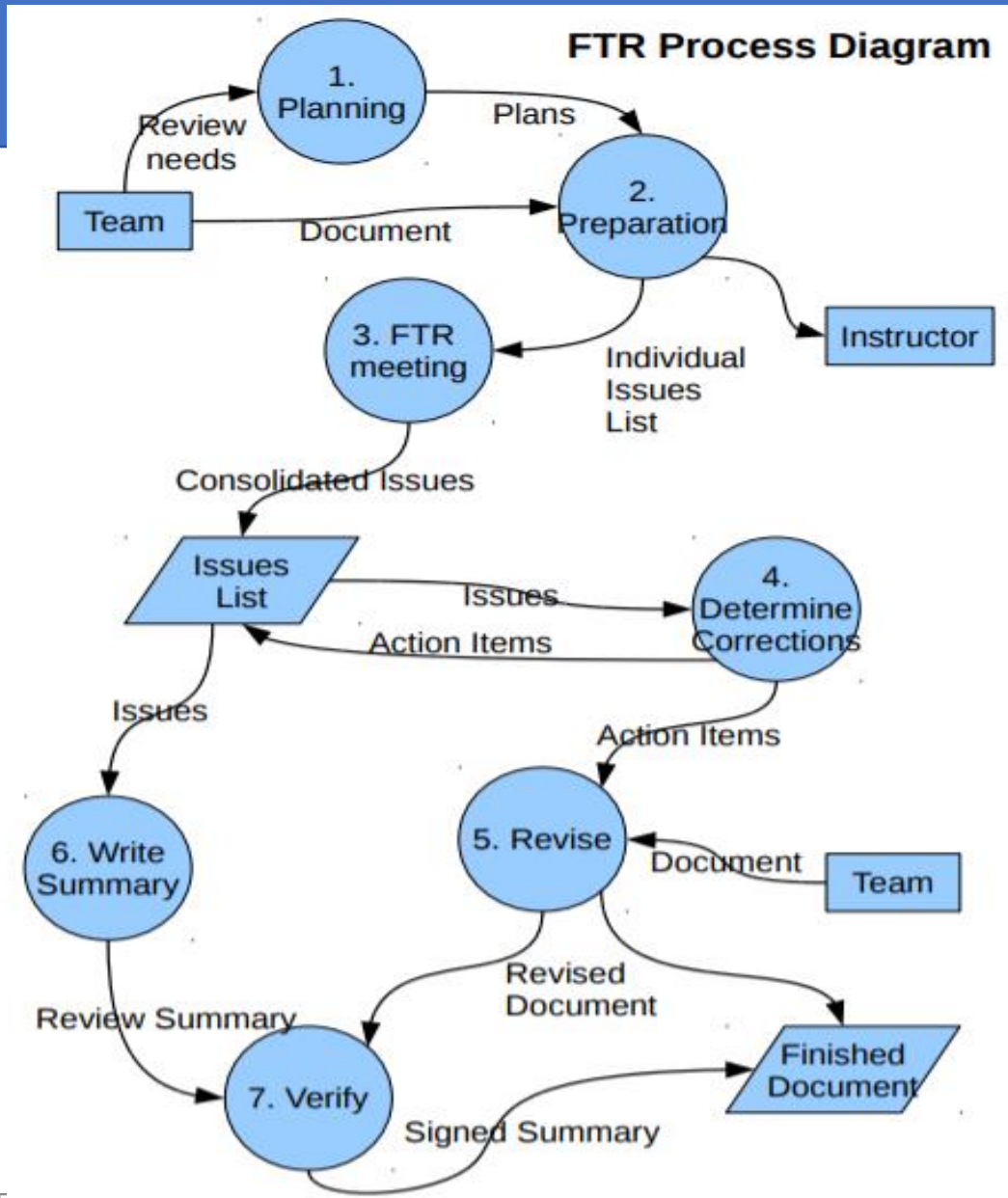
People involved in a review meeting

- Producer, review leader, 2 or 3 reviewers (one of them is recorder)

FTR process Diagram

Review meeting's constraints

- 3-5 people involved in a review
- Advanced preparation (no more than 2 hours for each person)
- The duration of the review meeting should be less than 2 hours
- Focus on a specific part of a software product



People involved in a review meeting

Before the Formal Technical Review (FTR)

The preparation of a review meeting

- A meeting agenda and schedule (by review leader)
- Review material and distribution (by the producer)
- Review in advance (by reviewers)

During the Formal Technical Review (FTR)

- Moderator, usually QA leader, is responsible for running the meeting and orchestrating how the issues are to be presented.
- Reviewers raise issues based on their advance preparation.
- Get the big problem issues on the table early. Don't get lost in minutely small clerical details.
- Identify problems, don't solve them.
- Focus on material, not the producer
- Elicit questions, don't answer them.
- Don't explain how your product works. An FTR is not a product demo.
- Watch the clock and keep momentum to get through all the issues in the alotted time.
- Recorder actively records all issues raised.

At the end of the FTR

- Meeting decisions
  - Accept the work product w/o further modification
  - Reject the work product due to errors
  - Accept the work under conditions (such as change and review)

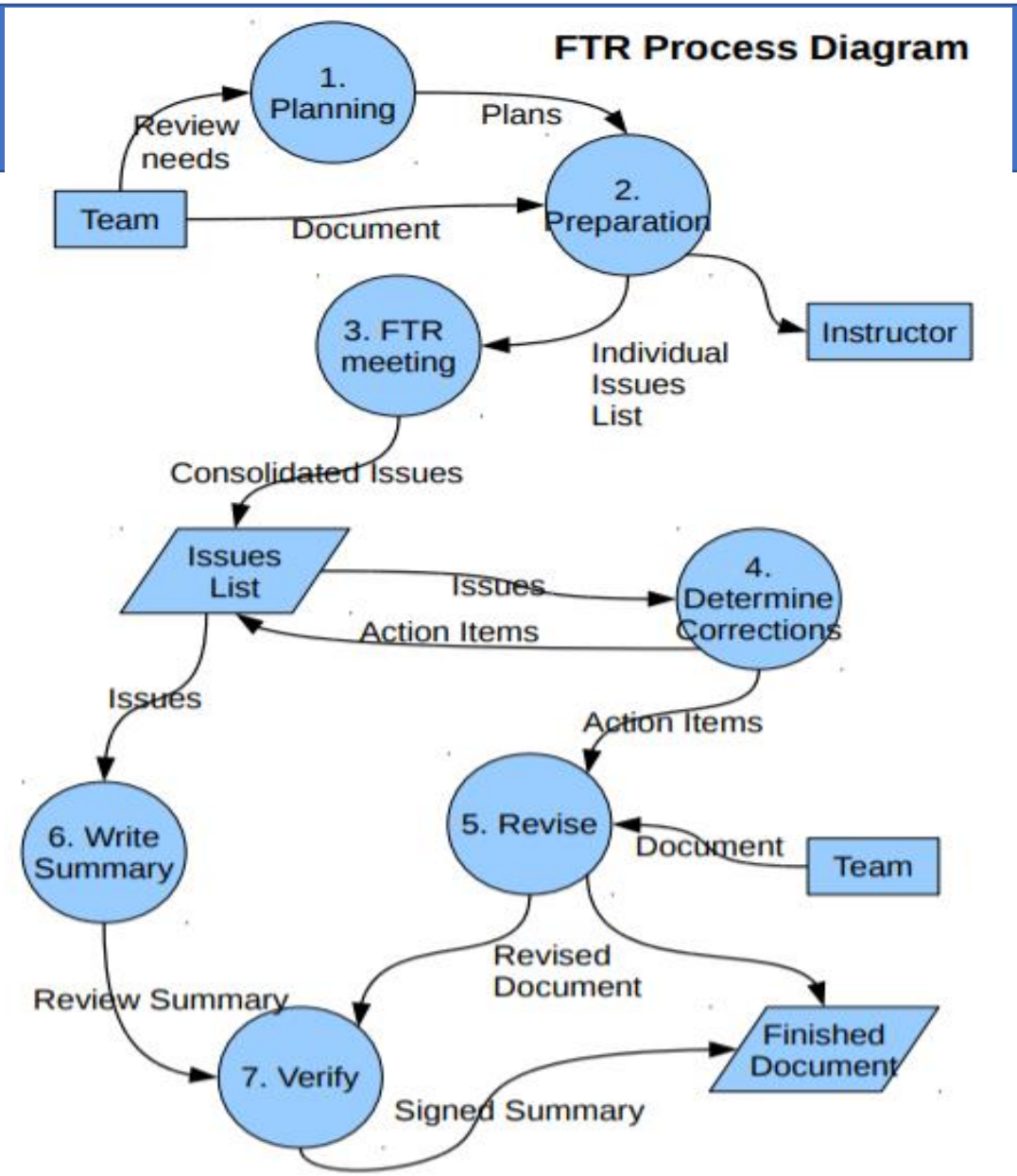
After the FTR

- Write up the issues list.
- Examine the issues raised, determine which to take action on.
- Assign each issue to someone as an action item and revise work product as needed.
- **Review issues list serves two purposes**
  - To identify problem areas in the project
  - To serve as an action item checklist (a follow-up procedure is needed)
- **Review summary report**
  - What was reviewed?
  - Who reviewed it?
  - What were the findings and conclusions?
- Have reviewers sign off on summary report, indicating proper corrective action has been taken for each issue.
- Include signed report in deliverable.

Minimum Guidelines for the FTR

- Review the product, not the producer
- Set an agenda and maintain it
- Limit debate and rebuttal
- Enunciate problem areas, but don’t attempt to solve every problem noted
- Take written notes
- Limit the number of participants and insist upon advance preparation
- Develop a checklist for each work product that is likely to be reviewed
- Allocate resources and time schedule for FTRs
- Conduct meaningful training for all reviewers
- Review your early reviews

FTR Process Diagram Revisit



Introduction to DevOps Tools

DevOps Tools

- **https://docs.docker.com/get-started/docker-overview/**
- https://www.jenkins.io/doc/
- https://kubernetes.io/docs/concepts/overview/